

分支任务lab2: gdb 调试页表查询过程

一、准备工作

1.1 编译带有调试信息的qemu

编译带有调试信息的qemu

```
1  # 复制 QEMU 源码目录到一个新的调试目录
2  cp -r ~/qemu-4.1.1 ~/qemu-4.1.1-debug
3  # 进入调试目录
4  cd ~/qemu-4.1.1-debug
5  # 解压 qemu-4.1.1.tar.xz 压缩包
6  tar xvJf qemu-4.1.1.tar.xz
7  # 将解压出的目录重命名为 qemu-4.1.1-debug (避免名字冲突)
8  mv qemu-4.1.1 qemu-4.1.1-debug
9  # 清理之前的编译结果
10 make distclean
11 # 配置, 启用调试信息
12 ./configure --target-list=riscv32-softmmu,riscv64-softmmu --enable-debug
13 # 编译
14 make -j$(nproc)
```

```
qemu-4.1.1/capstone/tests/test_arm.c
qemu-4.1.1/capstone/tests/test_arm64.c
qemu-4.1.1/capstone/tests/test_winkernel.cpp
qemu-4.1.1/exec.c
osaq@LAPTOP-QS28APKV:~$ cd ~/qemu-4.1.1
osaq@LAPTOP-QS28APKV:~/qemu-4.1.1$ cd ~
osaq@LAPTOP-QS28APKV:~$ mv qemu-4.1.1 qemu-4.1.1-debug
osaq@LAPTOP-QS28APKV:~$ ls
oslab  qemu-4.1.1-debug  qemu-4.1.1.tar.xz
osaq@LAPTOP-QS28APKV:~$ cd ~/qemu-4.1.1-debug
osaq@LAPTOP-QS28APKV:~/qemu-4.1.1-debug$ make distclean
rm -f *.timestamp
rm -f config.mak op-i386.h opc-i386.h gen-op-i386.h op-arm.h opc-arm.h gen-op-arm.h
rm -f qemu-options.def
rm -f *.msi
find . \( -name '*.so' -o -name '*.dll' -o -name '*.mo' -o -name '*.oda' \) -type f \
! -path ./roms/edk2/ArmPkg/Library/GccLto/liblto-aarch64.a \
! -path ./roms/edk2/ArmPkg/Library/GccLto/liblto-arm.a \
```

```

CC      riscv64-softmmu/target/riscv/translate.o
/home/osaq/qemu-4.1.1-debug/scripts/tracetool/__init__.py:456: SyntaxError:
if len(format) is 0:
/home/osaq/qemu-4.1.1-debug/scripts/tracetool/__init__.py:461: SyntaxError:
if len(backends) is 0:
CC      riscv64-softmmu/trace/generated-helpers.o
CC      riscv32-softmmu/trace/generated-helpers.o
LINK    riscv32-softmmu/qemu-system-riscv32
LINK    riscv64-softmmu/qemu-system-riscv64
osaq@LAPTOP-QS28APKV:~/qemu-4.1.1-debug$

```

编译成功！

注意这里不要执行 `sudo make install`！因为执行 `sudo make install` 会把 QEMU 安装到系统目录，覆盖系统自带版本，而调试时只需要在源码目录下直接运行生成的可执行文件即可，是不需要安装的。

1.2 修改makefile文件

接着我们修改ucore的Makefile，让它使用我们新编译的调试版QEMU

makefile文件修改处

```

1  debug:${(UCOREIMG)} ${(SWAPIMG)} ${(SFSIMG)}
2      ${(V)}/home/osaq/qemu-4.1.1-debug/riscv64-softmmu/qemu-system-riscv64 \
3          -machine virt \
4          -nographic \
5          -bios default \
6          -device loader,file=${(UCOREIMG)},addr=0x80200000\
7          -s -S

```

```

.DEFAULT_GOAL := TARGETS
.PHONY: qemu spike test
qemu: ${(UCOREIMG)} ${(SWAPIMG)} ${(SFSIMG)}
      ${(V)}${(QEMU)} \
          -machine virt \
          -nographic \
          -bios default \
          -device loader,file=${(UCOREIMG)},addr=0x80200000

debug: ${(UCOREIMG)} ${(SWAPIMG)} ${(SFSIMG)}
      ${(V)}/home/osaq/qemu-4.1.1-debug/riscv64-softmmu/qemu-system-riscv64 \
          -machine virt \
          -nographic \
          -bios default \
          -device loader,file=${(UCOREIMG)},addr=0x80200000\
          -s -S

gdb:
      riscv64-unknown-elf-gdb \
      -ex 'file bin/kernel' \
      -ex 'set arch riscv:rv64' \
      -ex 'target remote localhost:1234'

```

-S,-s参数解释

`-S` 表示 **启动时暂停 CPU，不自动执行 guest**，等待调试器连接或手动开始执行。

`-s` 是 `-gdb tcp::1234` 的简写，表示 **启动一个 GDB 服务器监听 TCP 1234 端口**，可以让 GDB 连接到 QEMU 进行远程调试。

组合使用时，QEMU 会 **启动后停在 CPU 起始点**，等待 GDB 连接并控制执行，有利于我们单步调试操作系统或内核代码。

1.3 启动三个终端（双重gdb调试）

终端1 启动qemu模拟器

主要是用于观察 QEMU 的标准输出（比如打印的内核日志）。

输入 make debug

```
osaq@LAPTOP-QS28APKV:~/oslab/lab2$ make debug
```

该命令运行效果为：QEMU 会启动，但因为 `-S` 参数，虚拟 CPU 会立即暂停，等待调试器发出命令；`-s` 参数会在本机开放 1234 端口，允许 GDB 远程连接。

终端2 附加调试 QEMU 自身源码

终端2作为调试 QEMU 模拟器的 GDB，在这里我们能看到 QEMU 在处理内存、TLB、页表等底层行为时的执行路径。

我们运行以下命令启动终端2

终端2启动相关命令

- 1 `pgrep -f qemu-system-riscv64`
- 2 `sudo gdb`
- 3 `(gdb) attach <PID>`
- 4 `(gdb) handle SIGPIPE nostop noprint`

命令解释

1. `pgrep -f qemu-system-riscv64`：找到 QEMU 进程 PID：
2. `sudo gdb`：启动 GDB
3. `(gdb) attach <PID>` 附加到该qemu进程
4. `(gdb) handle SIGPIPE nostop noprint`:设置信号处理（避免调试过程中被 SIGPIPE 打断）

```

osaq@LAPTOP-QS28APKV:~/oslab/lab2$ pgrep -f qemu-system-riscv64 [45/45]
17161
osaq@LAPTOP-QS28APKV:~/oslab/lab2$ sudo gdb
[sudo] password for osaq:
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".

```

```

(gdb) attach 17161 [23/45]
Attaching to process 17161
[New LWP 17162]
[New LWP 17163]
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
0x000078ff6dc5ecb6 in __poll (fds=0x5e40fd867410, nfd=7,
    timeout=<optimized out>, sigmask=0x0)
    at ../sysdeps/unix/sysv/linux/poll.c:44
44      ../sysdeps/unix/sysv/linux/poll.c: No such file or directory.
(gdb) handle SIGPIPE nostop noprint
Signal      Stop      Print     Pass to program Description
SIGPIPE     No        No        Yes       Broken pipe

```

handle SIGPIPE nostop noprint

看实验指导书中说最好输入这条指令，但一开始没有运行，想看看会出现什么问题，最初感觉并没有什么不同，调试过程没有遇到与之相关的问题，但是有一次调试时启动好终端2的时候忘记按c，然后直接在终端3运行了make gdb，由于这时qemu还处于暂停状态，这里显示远程 GDB 已经连接，但还没有下达继续指令

```

For help, type "help".
Type "apropos word" to search for commands related to "word".
Reading symbols from bin/kernel...
< q to quit, c to continue without paging--c
The target architecture is set to "riscv:rv64".
Remote debugging using localhost:1234

```

于是再回到终端2按下c，出现了如下输出

```
(gdb) c
Continuing.

Thread 1 "qemu-system-ris" received signal SIGPIPE, Broken pipe.
__libc_sendmsg (flags=0, msg=0x7fff88950eb0, fd=11)
  at ../sysdeps/unix/sysv/linux/sendmsg.c:28
28      ../sysdeps/unix/sysv/linux/sendmsg.c: No such file or direc
tory.
(gdb)
```

询问大模型得知 SIGPIPE 错误是因为 **GDB 尝试向已经关闭或不可用的 QEMU GDB 服务器发送继续指令，导致管道破裂**。为了避免这种情况的发生，我们需要在终端2运行前输入 `handle SIGPIPE nostop noprint`

由于我们计划先运行终端3（调试内核代码），终端3在我们设置的某个位置停下后，再回到终端2设置断点，从而防止终端3还没到我们设置的断点程序会莫名的停下，所以这里我们直接点击c运行

终端3 调试ucore内核

make gdb

然后我们在kern_init位置（uCore 内核的 C 语言入口函数，标志着内核初始化的正式开始，从这个位置开始，内核的各种初始化逻辑（如内存管理、设备驱动、进程调度）才真正执行）设置一个断点，接着按下c，内核代码会停在我们设的断点处，如下图所示

```
For help, type "help".
Type "apropos word" to search for commands related to "word".
Reading symbols from bin/kernel...
< q to quit, c to continue without paging--c
The target architecture is set to "riscv:rv64".
Remote debugging using localhost:1234
0x0000000000001000 in ?? ()
(gdb) b kern_init

Breakpoint 1 at 0xffffffffc02000d8: file kern/init/init.c, line 3
1.
(gdb) c
Continuing.

Breakpoint 1, kern_init () at kern/init/init.c:31
31      memset(edata, 0, end - edata);
(gdb)
```

二、尝试理解我们调试流程中涉及到的qemu的源码

要求：尝试理解我们调试流程中涉及到的qemu的源码，给出关键的调用路径，以及路径上一些关键的分支语句（不是让你只看分支语句），并通过调试演示某个访存指令访问的虚拟地址是如何在qemu的模拟中被翻译成物理地址的。

首先我们询问大模型对于观察qemu模拟cpu在虚拟地址空间访问一个虚拟地址是如何查找tlb以及页表应该设置哪些断点

大模型给出的回答（整合版本）：

CPU 在虚拟地址空间访问一个虚拟地址时的执行流程主要包括TLB 查找、页表查找和内存读写操作，主要关注这些操作的实现函数。

1、内存读写相关： `cpu_memory_rw`

2、TLB 相关： `cpu_translate`， `tlb_fill`

3、页表相关： `riscv_translate`、 `riscv_memory_rw`

按照上面的提示，我们设置相关的断点

```
(gdb) b cpu_memory_rw
```

```
(gdb) b cpu_translate
```

```
(gdb) b tlb_fill
```

```
(gdb) b riscv_translate
```

```
(gdb) b riscv_memory_rw
```

结果如下，发现只有一个断点 `tlb_fill` 设置成功了，其余断点都出现了”Function not defined.“的问题，可能是目标函数尚未加载或者函数名不对（由于这些函数对我们理解流程来说作用没有`tlb_fill`函数大，这个问题我们在最后部分再进行分析）

```
(gdb) b cpu_memory_rw
Function "cpu_memory_rw" not defined.
Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 1 (cpu_memory_rw) pending.
(gdb) b cpu_translate
Function "cpu_translate" not defined.
Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 2 (cpu_translate) pending.
(gdb) b tlb_fill
Breakpoint 3 at 0x5e40e7cee195: file /home/osaq/qemu-4.1.1-debug/accel/tcg/cputlb.c, line 870.
(gdb) b riscv_translate
Function "riscv_translate" not defined.
Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 4 (riscv_translate) pending.
(gdb) b riscv_memory_rw
Function "riscv_memory_rw" not defined.
Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 5 (riscv_memory_rw) pending.
(gdb) c
```

我们先了解一下QEMU 模拟的 CPU 访问一个虚拟地址的大致流程：

首先会先查 **软件 TLB**；如果 TLB 未命中，就会进入 `tlb_fill()`，由它根据当前特权级、访存类型（读/写/取指）去走 **RISC-V 页表遍历逻辑**（如 `get_physical_address`），完成虚拟地址到物理地址的转换，并把结果回填到 TLB 中；之后同类访问就直接命中 TLB，不再走页表。因此，**第一次缺失时的 `tlb_fill()` 正是观察 TLB 查找、页表遍历、权限检查与异常触发（如 page fault）的最佳位置**。所以这里我们首先重点关注 `tlb_fill()` 这个函数。

我们的**整体逻辑**是（和指导书中不同），不先去找某一条访存指令，而是直接设置断点，因为在我们设置的这些断点停下来的话，大概率就是遇到了这些访存指令，而不同访存指令的处理逻辑基本是相同的，所以这里我们不去管具体遇到了哪条访存指令，直接设置qemu中处理访存指令的函数中断进而分析。

打好断点后我们运行，发现在tlb_fill处发生了中断

```
(gdb) c
Continuing.
[Switching to Thread 0x78ff6d24c700 (LWP 17163)]

Thread 3 "qemu-system-ris" hit Breakpoint 3, tlb_fill (cpu=0x78ff6d24b7d0,
    addr=103633224089504, size=-1,
    access_type=(MMU_DATA_STORE | MMU_INST_FETCH | unknown: 4294967292),
    mmu_idx=24128, retaddr=255)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/cputlb.c:870
870      {
(gdb)
```

代码块

```
1  tlb_fill (cpu=0x78fff6d24b7d0, addr=103633224089504, size=-1,
2  access_type=(MMU_DATA_STORE | MMU_INST_FETCH | unknown:
    4294967292),mmu_idx=24128, retaddr=255)
```

我们发现这次 `tlb_fill` 的调用参数明显异常，说明它并不是一次正常的访存地址翻译：`addr` 是一个远超 uCore 正常地址空间的巨大虚拟地址，`size=-1` 不符合任何合法访存宽度，`access_type` 同时包含数据写和取指且带有未知高位标志，`mmu_idx=24128` 也远超正常特权级对应的取值范围，这些都表明当前 CPU 可能处在异常处理或特权级切换（如 `sret`）后的不稳定状态，或者命中了 QEMU TCG 的边缘/伪访存路径；该 `tlb_fill` 可能是异常或 JIT 内部触发的填充尝试，而不是一次标准的“虚拟地址 → 物理地址”的 TLB miss 翻译过程。

于是我们舍弃这次 `tlb_fill` 中断，继续向后分析找到正常的 `tlb` 调用。

接着我们试着输入单步调试，发现调用了很多次 `tlb_fill`，但每次调用除了一般参数，如访存类 `access_type` 和指令获取 `mmu_idx` 的值可能也会略有不同，我们选取了不同访存类型的 `tlb_fill` 分别进行了分析。

这里我们来到了访存类型为 `MMU_INST_FETCH` 的 `tlb` 函数 (`mmu_index=3`)，如下所示


```

0x00005a86c1c461c3 in tlb_fill (cpu=0x5a86d538e590, addr=4096,
    size=0, access_type=MMU_INST_FETCH, mmu_idx=3, retaddr=0)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/cputlb.c:871
871      CPUClass *cc = CPU_GET_CLASS(cpu);
(gdb) p cpu->as
$1 = (AddressSpace *) 0x5a86d53a4a50
(gdb) p cpu->memory
$2 = (MemoryRegion *) 0x5a86d531df00
(gdb) x/10i $rip
=> 0x5a86c1c461c3 <tlb_fill+46>:
    lea     0x569af6(%rip),%r8          # 0x5a86c21afcc0 <__func__.281
16>
    0x5a86c1c461ca <tlb_fill+53>:      mov     $0x367,%ecx
    0x5a86c1c461cf <tlb_fill+58>:      lea     0x569972(%rip),%rdx        # 0x5a86c21afb48
    0x5a86c1c461d6 <tlb_fill+65>:      lea     0x5697d1(%rip),%rsi        # 0x5a86c21af9ae
    0x5a86c1c461dd <tlb_fill+72>:      mov     %rax,%rdi
    0x5a86c1c461e0 <tlb_fill+75>:      callq   0x5a86c1f4b7a6 <object_class_dynamic_cast_assert>
    0x5a86c1c461e5 <tlb_fill+80>:      mov     %rax,-0x8(%rbp)
    0x5a86c1c461e9 <tlb_fill+84>:      mov     -0x8(%rbp),%rax
    0x5a86c1c461ed <tlb_fill+88>:      mov     0x148(%rax),%r10
    0x5a86c1c461f4 <tlb_fill+95>:      mov     -0x2c(%rbp),%edi
(gdb) i r pc
pc                0x5a86c1c461c3      0x5a86c1c461c3 <tlb_fill+46>
(gdb) p/x cpu->env.pc
There is no member named env.
(gdb) print/x cpu
$3 = 0x5a86d538e590
(gdb) info args
cpu = 0x5a86d538e590
addr = 4096
size = 0
access_type = MMU_INST_FETCH

```

tlb_fill

```

1  tlb_fill(
2      cpu=0x5a86d538e590,          # CPU指针 (正常)
3      addr=4096,                  # 地址: 0x1000 (可疑!)
4      size=0,                    # 大小: 0字节 (无效!)
5      access_type=MMU_INST_FETCH, # 访问类型: 指令获取
6      mmu_idx=3,                  # MMU索引: 3 (机器模式)
7      retaddr=0                   # 返回地址: 0 (无效!)
8  )

```

分析可知当前 GDB 停在 `tlb_fill` 函数内部，PC 地址为 `0x5a86c1c461c3`，表示 CPU 正在处理一次虚拟地址 `0x1000` 的指令取值 (`MMU_INST_FETCH`) TLB 填充操作。`cpu->as` 指向该 CPU 的 `AddressSpace`，`cpu->memory` 指向其 `MemoryRegion`，说明 TLB 填充时需要访问对应的地址空间和内存区域。反汇编显示函数正在进行对象类型检查和动态转换

(`object_class_dynamic_cast_assert`)，用于确保 `cpu` 对象的正确类型。`size=0` 表示当前访存大小无效，属于指令取值类型下的特例，而 `$3` 和 `cpu` 参数确认了传入的 CPU 对象指

针。整体来看，这次停靠点正处于 QEMU 模拟 CPU 处理指令取值虚拟地址、准备更新 TLB 的慢路径逻辑中。

接着我们又使用单步调试来到了 `access_type` 为 `MMU_DATA_STORE` 的 `tlb_fill` 函数

```
(gdb) si
0x0000569ad272a1a1      870  {
(gdb) si
0x0000569ad272a1a5      870  {
(gdb) si
0x0000569ad272a1a9      870  {
(gdb) s
871      CPUClass *cc = CPU_GET_CLASS(cpu);
(gdb) si
0x0000569ad272a1bb      871      CPUClass *cc = CPU_GET_CLASS(cpu);
(gdb) si
0x0000569ad272a1be      871      CPUClass *cc = CPU_GET_CLASS(cpu);
(gdb) si
object_get_class (obj=0xffffffffc0204ff8) at qom/object.c:907
907  {
(gdb) si
0x0000569ad2a2f94f      907  {
(gdb) si
0x0000569ad2a2f950      907  {
(gdb) si
0x0000569ad2a2f953      907  {
(gdb) si
908      return obj->class;
(gdb) si
0x0000569ad2a2f95b      908      return obj->class;
(gdb) si
909  }
(gdb) si
0x0000569ad2a2f95f      909  }
(gdb) si
0x0000569ad272a1c3 in tlb_fill (cpu=0x569b12d18590,
    addr=18446744072637927416, size=8,
    access_type=MMU_DATA_STORE, mmu_idx=1,
    retaddr=135398113042803)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/cputlb.c:871
871      CPUClass *cc = CPU_GET_CLASS(cpu);
(gdb)
```

```
Platform Name       : QEMU Virt Machine
Platform HART Features : RV64ACDFIMSU
Platform Max HARTs    : 8
Current Hart         : 0
Firmware Base        : 0x80000000
Firmware Size        : 112 KB
Runtime SBI Version   : 0.1

PMP0: 0x0000000000000000-0x0000000000000001ffff (A)
PMP1: 0x0000000000000000-0xffffffffffffffff (A,R,W,X)

For help, type "help".
Type "apropos word" to search for commands related to "word".
Reading symbols from bin/kernel...
< q to quit, c to continue without paging -c
The target architecture is set to "riscv:rv64".
Remote debugging using localhost:1234
0x00000000000001000 in ?? ()
(gdb) b kern_init

Breakpoint 1 at 0xffffffffc02000d8: file kern/init/init.c, line 3
1.
(gdb) c
Continuing.

Breakpoint 1, kern_init () at kern/init/init.c:31
31      memset(edata, 0, end - edata);
(gdb) c
Continuing.
```

接下来开始单步调试，其中我们遇到了函数 `store_helper`

```
store_helper (big_endian=false, size=8, retaddr=135398113042803,
    oi=49, val=2147486210, addr=18446744072637927416,
    env=0x569b12d20fa0)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/cputlb.c:1524
1524      index = tlb_index(env, mmu_idx, addr);
(gdb) n
1525      entry = tlb_entry(env, mmu_idx, addr);
(gdb) n
1527      tlb_addr = tlb_addr_write(entry) & ~TLB_INVALID_MASK;
(gdb) n
1531      if (unlikely(tlb_addr & ~TARGET_PAGE_MASK)) {
(gdb) n
1563      if (size > 1
(gdb) n
1564          && unlikely((addr & ~TARGET_PAGE_MASK) + size - 1
(gdb) n
1606      do_aligned_access:
(gdb) n
1607      haddr = (void *)((uintptr_t)addr + entry->addend);
(gdb) n
1608      switch (size) {
(gdb) n
1627          if (big_endian) {
(gdb) n
1630              stq_le_p(haddr, val);
(gdb) n
1632          break;
(gdb) n
```

```

1: store_helper (big_endian=false, size=8, retaddr=135398113042803,oi=49,
   val=2147486210, addr=18446744072637927416,env=0x569b12d20fa0)
2:         index = tlb_index(env, mmu_idx, addr);
3:         entry = tlb_entry(env, mmu_idx, addr);
4:         tlb_addr = tlb_addr_write(entry) & ~TLB_INVALID_MASK;
5:         if (unlikely(tlb_addr & ~TARGET_PAGE_MASK)) {
6:             if (size > 1
7:                 && unlikely((addr & ~TARGET_PAGE_MASK) + size - 1
8: do_aligned_access:
9:                 haddr = (void *)((uintptr_t)addr + entry->addend);
10:                switch (size) {
11:                    if (big_endian) {
12:                        stq_le_p(haddr, val);
13:                        break;

```

store_helper函数的主要作用是在 QEMU 中模拟一次 guest 的内存写操作，通过 TLB 将虚拟地址快速转换为宿主可访问的地址并完成实际写入。

当 guest 执行一条 store 指令时，QEMU 进入 store_helper，先根据当前 MMU 上下文和虚拟地址用 tlb_index / tlb_entry 查找对应的 TLB 项，计算出写访问的 tlb_addr 并检查其有效性；如果 TLB 项无效或跨页（如写入大小超过当前页剩余空间），就走慢路径触发对齐检查或最终调用 tlb_fill 做页表遍历；若检查通过，则利用 entry->addend 将 guest 虚拟地址直接加偏移转换成宿主地址 haddr，最后根据访问大小和字节序（这里是小端、8 字节）调用 stq_le_p 将数据真正写入宿主内存，从而完成一次“虚拟地址 → 物理内存写”的模拟。

然后这里来到了函数helper_le_stq_mmu

```

1673 helper_le_stq_mmu (env=0x569b12d20fa0, addr=18446744072637927416,
   val=2147486210, oi=49, retaddr=135398113042803)
   at /home/osaq/qemu-4.1.1-debug/accel/tcg/cputlb.c:1673
1673 }

```

helper_le_stq_mmu

```

1: helper_le_stq_mmu (env=0x569b12d20fa0, addr=18446744072637927416,
2:   val=2147486210, oi=49, retaddr=135398113042803)
3:   }

```

helper_le_stq_mmu 是 QEMU 在 TLB 未命中或需要走慢路径时，用于模拟一次小端 8 字节（quadword）的内存写入，并带 MMU/页表检查的辅助函数。

接下来我们又再次在这个异常的tlb_fill处停下了，我们尝试仍然使用单步调试查看后续执行逻辑

```

Thread 3 "qemu-system-ris" hit Breakpoint 1, tlb_fill (
  cpu=0x7b24cffffe2a0, addr=95224035676064, size=-1,
  access_type=(MMU_DATA_STORE | MMU_INST_FETCH | unknown: 4294967
292), mmu_idx=22171, retaddr=63)
  at /home/osaq/qemu-4.1.1-debug/accel/tcg/cputlb.c:870
870     {
(gdb) n
871         CPUClass *cc = CPU_GET_CLASS(cpu);
(gdb) n
878         ok = cc->tlb_fill(cpu, addr, size, access_type, mmu_idx
, false, retaddr);
(gdb) n
879         assert(ok);
(gdb) n
880     }
(gdb) n
get_page_addr_code (env=0x569b12d20fa0, addr=18446744072637913716)
  at /home/osaq/qemu-4.1.1-debug/accel/tcg/cputlb.c:1033
1033         index = tlb_index(env, mmu_idx, addr);
(gdb) n
1034         entry = tlb_entry(env, mmu_idx, addr);
(gdb) n
1036         assert(tlb_hit(entry->addr_code, addr));
(gdb) n
1039         if (unlikely(entry->addr_code & (TLB_RECHECK | TLB_MMIO
))) {
(gdb) n
1051         p = (void *)((uintptr_t)addr + entry->addend);
(gdb) n
1052         return qemu_ram_addr_from_host_nofail(p);
(gdb) n
1053     }

```

tlb_fill

```

1  tlb_fill (cpu=0x7b24cffffe2a0, addr=95224035676064, size=-1,access_type=
(MMU_DATA_STORE | MMU_INST_FETCH | unknown: 4294967292),mmu_idx=22171,
retaddr=63)
2      {
3          CPUClass *cc = CPU_GET_CLASS(cpu);
4          ok = cc->tlb_fill(cpu, addr, size, access_type, mmu_idx, false,
retaddr);
5          assert(ok);
6      }
7  get_page_addr_code (env=0x569b12d20fa0, addr=18446744072637913716)
8      index = tlb_index(env, mmu_idx, addr);
9      entry = tlb_entry(env, mmu_idx, addr);
10     assert(tlb_hit(entry->addr_code, addr));
11     if (unlikely(entry->addr_code & (TLB_RECHECK | TLB_MMIO))) {
12         p = (void *)((uintptr_t)addr + entry->addend);
13         return qemu_ram_addr_from_host_nofail(p);
14     }

```

可以看到这里调用了`tlb_fill`和`get_page_addr_code`两个函数。

TLB 未命中，`tlb_fill` 从 `CPUClass` 中取出具体的 CPU（这里是 RISC-V）的 `tlb_fill` 实现，将虚拟地址、访问类型和 MMU 上下文传进去，由该实现完成权限检查、物理地址计算或异常触发；`assert(ok)` 表示若映射无法建立，函数不会正常返回，而是已经通过异常流程中断执行。

`get_page_addr_code` 用于 **取指路径中通过 TLB 快速把虚拟指令地址转换为宿主可访问的 RAM 地址**，它先根据 `addr` 和 `mmu_idx` 定位对应的 TLB 项并断言命中，然后检查该项是否需要重新检查或是否是 MMIO；若是普通 RAM 映射，就用 `addr + entry->addend` 得到宿主地址并转换为 QEMU 的 RAM 地址返回，从而避免再次走页表，是典型的 TLB 快路径实现。

接下来调用了`tb_lookup`，`tb_lookup_cpu_state`，`helper_lookup_tb_ptr`这些函数

```
(gdb) n
tb_htable_lookup (cpu=0x569b12d18590, pc=18446744072637913716,
    cs_base=0, flags=24577, cf_mask=4278714368)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:340
340         if (phys_pc == -1) {
(gdb) si
0x0000569ad274f2ef      340         if (phys_pc == -1) {
(gdb) n
343         desc.phys_page1 = phys_pc & TARGET_PAGE_MASK;
(gdb) n
344         h = tb_hash_func(phys_pc, pc, flags, cf_mask, *cpu->tra
ce_dstate);
(gdb) n
345         return qht_lookup_custom(&tb_ctx.htable, &desc, h, tb_l
ookup_cmp);
(gdb) n
346     }
(gdb) n
tb_lookup_cpu_state (cpu=0x569b12d18590, pc=0x7b24cffffe410,
    cs_base=0x7b24cffffe408, flags=0x7b24cffffe404,
    cf_mask=4278714368)
    at /home/osaq/qemu-4.1.1-debug/include/exec/tb-lookup.h:44
44         if (tb == NULL) {
(gdb) n
45             return NULL;
(gdb) n
49     }
(gdb) n
helper_lookup_tb_ptr (env=0x569b12d20fa0)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/tcg-runtime.c:155
155         if (tb == NULL) {
(gdb) n
156             return tcg_ctx->code_gen_epilogue;
(gdb) n
164     }
(gdb) n
0x00007b24d46e618f in code_gen_buffer ()
```

`tb_lookup`, `tb_lookup_cpu_state`, `helper_lookup_tb_ptr`

```
1  tb_lookup (cpu=0x569b12d18590, pc=18446744072637913716,hs_table.base=0,
    flags=24577, cf_mask=4278714368)
2      if (phys_pc == -1) {
```

```

3         desc.phys_page1 = phys_pc & TARGET_PAGE_MASK;
4         h = tb_hash_func(phys_pc, pc, flags, cf_mask, *cpu->trace_dstate);
5         return qht_lookup_custom(&tb_ctx.htable, &desc, h, tb_lookup_cmp);
6     }
7     tb_lookup_cpu_state (cpu=0x569b12d18590,
pc=0x7b24cffe410,cs_base=0x7b24cffe408,
flags=0x7b24cffe404,cf_mask=4278714368)
8         if (tb == NULL) {
9             return NULL;
10        }
11    helper_lookup_tb_ptr (env=0x569b12d20fa0)
12        if (tb == NULL) {
13            return tcg_ctx->code_gen_epilogue;
14        }

```

tb_lookup: 它的作用是 根据当前 PC（以及对应的物理地址）、CPU 标志和控制流掩码，在 TB 哈希表中查找是否已有可复用的翻译块；它先用物理页地址参与计算哈希值，再通过 `tb_hash_func` 和 `qht_lookup_custom` 在 TB 缓存中查找匹配的 TB，用于实现“相同指令流不重复翻译”的加速。

tb_lookup_cpu_state: 它是对 `tb_lookup` 的一层封装，它从 CPU 状态中提取当前 PC、cs_base、flags 等执行上下文信息，并据此尝试查找对应的 TB；如果返回 `NULL`，表示当前 CPU 状态下没有可用的 TB，需要进入翻译流程重新生成。

helper_lookup_tb_ptr: 该函数运行在 TCG 执行路径中，用来 在运行时获取下一个要执行的 TB 的入口地址；如果查找不到 TB，就返回 `code_gen_epilogue`，从而回到 QEMU 主循环触发翻译或异常处理，这个函数是 JIT 执行与 C 层调度之间的关键衔接点。

最后我们再单步调试，来到了code_gen_buffer函数

```

(gdb) n
0x00007b24d46e618f in code_gen_buffer ()
(gdb) n
Single stepping until exit from function code_gen_buffer,
which has no line number information.
0x00007b24d46e6016 in ?? ()
(gdb) n
Cannot find bounds of current function
(gdb) n
Cannot find bounds of current function

```

可以看到，这里出现了一个奇怪的输出

code_gen_buffer

1 (gdb) n

```

2  0x00007b24d46e618f in code_gen_buffer ()
3  (gdb) n
4  Single stepping until exit from function code_gen_buffer,
5  which has no line number information.
6  0x00007b24d46e6016 in ?? ()
7  (gdb) n
8  Cannot find bounds of current function
9  (gdb) n
10 Cannot find bounds of current function

```

这是因为我们在用 GDB 单步调试时进入了 QEMU TCG 动态生成的 JIT 代码区域：

`code_gen_buffer` 会在运行时生成宿主机（如 x86-64）的机器码，这些代码不在源码中、没有符号信息也没有函数边界，因此当 GDB 执行到这里时既无法定位源码也无法判断函数范围，就会出现“Single stepping until exit from function `code_gen_buffer`”和“Cannot find bounds of current function”的提示；这属于 QEMU TCG 调试中的正常现象而非错误。

后来我们单步调试过程中遇到了这个函数

```

0x0000569ad274fda4 in cpu_exec (cpu=0x569b12d18590)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:689
689         if (sigsetjmp(cpu->jmp_env, 0) != 0) {
(gdb)

```

cpu_exec

```

1  0x0000569ad274fda4 in cpu_exec (cpu=0x569b12d18590)
2  at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:689
3  689         if (sigsetjmp(cpu->jmp_env, 0) != 0) {

```

`cpu_exec` 是 QEMU TCG/CPU 执行的主循环，它从 `CPUState` 里获取当前 CPU 状态，通过 **JIT 或解释器** 执行指令，并且管理异常、中断和翻译块（TB）的执行（我们在后面 lab5 的分支中对此有更加详细的分析）

在这里，我们可以看到 `cpu_exec` 函数的参数为 `cpu=0x569b12d18590`，即指的是当前的 CPU 对象

cpu_exec

```

1  if (sigsetjmp(cpu->jmp_env, 0) != 0) {
2      // 异常处理
3  }

```

这里 QEMU 使用 `sigsetjmp` / `longjmp` 机制处理 **CPU 异常或 MMU/TLB 访问错误**，当 CPU 执行指令触发异常（如页错误、未对齐访问）时，会用 `longjmp` 跳回这里；`jmp_env` 用于保存当前

执行环境。

返回值如果是 0，就正常执行，继续下方循环；否则非 0，由 longjmp 触发，进入异常处理路径

当前到这里说明 QEMU 已完成 JIT 代码执行或触发异常，

cpu_exec 是 C 层调试的合适入口，不再进入动态生成的机器码。

接下来来到了函数 tb_find

```
tb_find (cpu=0x569b12d18590, last_tb=0x0, tb_exit=0,
         cf_mask=524288)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:407
407         if (tb == NULL) {
(gdb) n
408             mmap_lock();
(gdb) n
409             tb = tb_gen_code(cpu, pc, cs_base, flags, cf_mask);
(gdb) n
410             mmap_unlock();
(gdb) n
412             atomic_set(&cpu->tb_jump_cache[tb_jump_cache_hash_fun
c(pc)], tb);
(gdb) n
419             if (tb->page_addr[1] != -1) {
(gdb) n
424                 if (last_tb) {
(gdb) n
427                     return tb;
(gdb) n
428             }
(gdb) n
cpu_exec (cpu=0x569b12d18590)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:732
```

tb_find

```
1  tb_find (cpu=0x569b12d18590, last_tb=0x0, tb_exit=0, cf_mask=524288)
2      if (tb == NULL) {
3          mmap_lock();
4          tb = tb_gen_code(cpu, pc, cs_base, flags, cf_mask);
5          mmap_unlock();
6          atomic_set(&cpu->tb_jump_cache[tb_jump_cache_hash_func(pc)], tb);
7          if (tb->page_addr[1] != -1) {
8              if (last_tb) {
9                  return tb;
10             }
11  cpu_exec (cpu=0x569b12d18590)
```

tb_find 主要作用为在 QEMU TCG 中寻找或生成可执行的翻译块（TB）并准备执行。执行逻辑如下

- 1、`tb_find` 先尝试在 TB 缓存中查找当前 CPU 和 PC 对应的 TB，如果存在就直接返回，避免重复翻译。
- 2、如果找不到 (`tb == NULL`)，则加锁 `mmap_lock()` 后调用 `tb_gen_code` 根据当前 PC、CPU 状态、标志和控制流掩码生成新的 TB（包含翻译后的宿主指令），生成完成后释放锁。
- 3、新生成的 TB 会写入 `tb_jump_cache`，加速下次相同 PC 的跳转。
- 4、条件检查如 `tb->page_addr[1] != -1` 和 `last_tb` 用于处理分页边界或 TB 链接优化。
- 5、最终返回 TB 后，`cpu_exec` 会执行该 TB，驱动 CPU 模拟的指令流运行，实现从虚拟地址到宿主指令的动态翻译与执行。

最后我们来简单梳理一下 TLB 查找的逻辑：

主要通过调用 `tlb_entry(env, mmu_idx, addr)` 获取条目

如果 TLB 命中，直接得到物理地址，访问内存；

如果 TLB miss，就调用 `tlb_fill`，进而判断是否缺页进行对应的处理。

三、页表翻译过程

要求：单步调试页表翻译的部分，解释一下关键的操作流程。

询问大模型，我们了解到 qemu 处理页表翻译主要就是在 `get_physical_address()` 里完成的，该函数的作用主要是 **将一个 guest 虚拟地址通过页表遍历转换成物理地址**，它会根据当前 CPU 特权级和访问类型，检查页表项的有效性和权限，如果访问合法则返回对应物理地址，如果无效或权限不够则触发缺页或权限异常，是模拟 RISC-V MMU 核心的地址翻译函数。**整个的调用链如下所示：**

TCG 执行访存

↓

softmmu 发现需要 VA → PA

↓

`tlb_fill()`

↓

`riscv_cpu_tlb_fill()`

↓

`get_physical_address()` ← ★核心函数

如果 `get_physical_address()` 返回失败：

`get_physical_address()`

↓ fail

`riscv_cpu_tlb_fill()`

↓

riscv_raise_exception()

↓

riscv_cpu_do_interrupt()

↓

进入 uCore trap handler，由内核（uCore）真正的处理缺页

为了调试查看页表翻译的具体逻辑，我们选择在**tlb_fill**、**riscv_cpu_tlb_fill**、**get_physical_address**、**riscv_raise_exception**这四个关键的函数处打下断点。

首先我们仍然是先在终端3中打下断点b kern_init,接着运行到该断点处，再进入终端2，按下ctrl+c后输入

b tlb_fill

b riscv_cpu_tlb_fill

b get_physical_address

b riscv_raise_exception

如下所示

```
(gdb) b tlb_fill
Breakpoint 1 at 0x5b3e321b9195: file /home/osaq/qemu-4.1.1-debug/accel/tcg/cputlb.c, line 870.
(gdb) b riscv_cpu_tlb_fill
Breakpoint 2 at 0x5b3e32271303: file /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c, line 438.
(gdb) b get_physical_address
Breakpoint 3 at 0x5b3e322706f1: file /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c, line 158.
(gdb) b riscv_raise_exception
Breakpoint 4 at 0x5b3e3226f90a: file /home/osaq/qemu-4.1.1-debug/target/riscv/op_helper.c, line 30.
(gdb) c
Continuing.
```

接着我们回到终端3，输入c运行，结果如下，在get_physical_address函数处停下了，

```
Thread 1 "qemu-system-ris" hit Breakpoint 3, get_physical_address
(env=0x5c5c8a575ffd <trace_qemu_mutex_unlock+41>,
physical=0x7ffe2a9d8750, prot=0x5c5cc7756a80,
addr=101552529024910, access_type=23644, mmu_idx=-1972689064)
at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:158
158      {
```

然后我们进行单步调试，仔细查看这个函数具体的执行逻辑

```

Thread 1 "qemu-system-ris" hit Breakpoint 3, get_physical_address
(env=0x5b92f5452ffd <trace_qemu_mutex_unlock+41>,
physical=0x7ffee85b44c0, prot=0x5b93096f0a80,
addr=100686739601294, access_type=23442, mmu_idx=-178718888)
at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:158
158      {
(gdb) n
163      int mode = mmu_idx;
(gdb) n
165      if (mode == PRV_M && access_type != MMU_INST_FETCH) {
(gdb) n
166          if (get_field(env->mstatus, MSTATUS_MPRV)) {
(gdb) n
171          if (mode == PRV_M || !riscv_feature(env, RISCV_FEATURE_
MMU)) {
(gdb) n
172              *physical = addr;
(gdb) n
173              *prot = PAGE_READ | PAGE_WRITE | PAGE_EXEC;
(gdb) n
174              return TRANSLATE_SUCCESS;
(gdb) n
353      }
(gdb) n

```

get_physical_address

```

1  get_physical_address(env=0x5b92f5452ffd <trace_qemu_mutex_unlock+41>,
2  physical=0x7ffee85b44c0, prot=0x5b93096f0a80,
3  addr=100686739601294, access_type=23442, mmu_idx=-178718888)
4      {
5          int mode = mmu_idx;
6  (gdb) n
7  165          if (mode == PRV_M && access_type != MMU_INST_FETCH) {
8  (gdb) n
9  166          if (get_field(env->mstatus, MSTATUS_MPRV)) {
10 (gdb) n
11  171          if (mode == PRV_M || !riscv_feature(env, RISCV_FEATURE_MMU)) {
12 (gdb) n
13  172              *physical = addr;
14 (gdb) n
15  173              *prot = PAGE_READ | PAGE_WRITE | PAGE_EXEC;
16 (gdb) n
17  174              return TRANSLATE_SUCCESS;
18 (gdb) n
19  353      }
20 (gdb) n

```

`get_physical_address()` 函数首先根据 `mmu_idx` 推断当前特权级 `mode`，在机器模式（PRV_M）或 MMU 未启用时直接将虚拟地址视为物理地址并赋予读写执行权限返回成功；若启用了 MMU 且不是简单机器模式访问，则会进一步检查 MPRV 标志、页表和权限，执行完整页表遍历与访问检查，以确保访问合法，否则会触发缺页或权限异常，从而模拟真实 CPU 的地址转换和异常处理行为。

在我们这里，代码显示当前 CPU 特权级或 `mmu_idx` 被识别为机器模式（PRV_M）或 MMU 未启用（可能是因为我们现在正处于**内核启动早期**），因此函数直接把传入的虚拟地址 `addr` 当作物理地址返回，同时赋予读写执行权限 `PAGE_READ | PAGE_WRITE | PAGE_EXEC` 并返回 `TRANSLATE_SUCCESS`，说明此时没有触发页表翻译或权限检查，模拟了 RISC-V 在机器模式或 MMU 禁用时的直接物理访问行为。

我们继续输入 c 单步调试，结果如下，进入了 `riscv_cpu_get_phys_page_debug` 这个函数

```
(gdb) n
riscv_cpu_get_phys_page_debug (cs=0x5b930979c590, addr=0)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:389
389         if (get_physical_address(&cpu->env, &phys_addr, &prot,
(gdb) c
Continuing.
```

为了了解这个函数的具体执行逻辑，我们还是单步调试，结果如下

```

riscv_cpu_get_phys_page_debug (cs=0x5b930979c590,
                                addr=18446744072637906944)
    at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:389
389         if (get_physical_address(&cpu->env, &phys_addr, &prot,
addr, 0, mmu_idx)) {
(gdb) n
392         return phys_addr;
(gdb) n
393     }
(gdb) n
cpu_get_phys_page_attrs_debug (cpu=0x5b930979c590,
                                addr=18446744072637906944, attrs=0x7ffee85b4618)
    at /home/osaq/qemu-4.1.1-debug/include/qom/cpu.h:608
608     }
(gdb) n
cpu_memory_rw_debug (cpu=0x5b930979c590,
                     addr=18446744072637906944, buf=0x7ffee85b4744 "", len=64,
                     is_write=0) at /home/osaq/qemu-4.1.1-debug/exec.c:3977
3977         asidx = cpu_asidx_from_attrs(cpu, attrs);
(gdb) n
3979         if (phys_addr == -1)
(gdb) n
3981         l = (page + TARGET_PAGE_SIZE) - addr;
(gdb) n
3982         if (l > len)
(gdb) n
3983             l = len;
(gdb) n
3984         phys_addr += (addr & ~TARGET_PAGE_MASK);
(gdb) n
3985         if (is_write) {
(gdb) n
3989             address_space_rw(cpu->cpu_ases[asidx].as, phys_
addr,

```

代码块

```

1  riscv_cpu_get_phys_page_debug (cs=0x5b930979c590, addr=18446744072637906944)
2      if (get_physical_address(&cpu->env, &phys_addr, &prot, addr, 0,
mmu_idx)) {
3      return phys_addr;
4      }

```

`riscv_cpu_get_phys_page_debug` 的作用是在调试模式下获取 guest 虚拟地址对应的物理页地址：它调用 `get_physical_address` 将传入的虚拟地址 `addr` 翻译为物理地址 `phys_addr` 并获取访问权限 `prot`，如果翻译成功（函数返回非零），就直接返回物理地址，用于调试观察或辅助打印虚拟地址对应的宿主内存位置。

可以看到后续还调用了 `cpu_get_phys_page_attrs_debug` 和 `cpu_memory_rw_debug` 这两个函数。

```

1  riscv_cpu_get_phys_page_debug (cs=0x5b930979c590, addr=18446744072637906944)
2      if (get_physical_address(&cpu->env, &phys_addr, &prot, addr, 0,
3      mmu_idx)) {
4          return phys_addr;
5      }
6  cpu_get_phys_page_attrs_debug (cpu=0x5b930979c590, addr=18446744072637906944,
7      attrs=0x7fffee85b4618)
8      }
9  cpu_memory_rw_debug (cpu=0x5b930979c590, addr=18446744072637906944,
10     buf=0x7fffee85b4744 "", len=64, is_write=0)
11     asidx = cpu_asidx_from_attrs(cpu, attrs);
12     if (phys_addr == -1)
13     l = (page + TARGET_PAGE_SIZE) - addr;
14     if (l > len)
15         l = len;
16     phys_addr += (addr & ~TARGET_PAGE_MASK);
17     if (is_write) {
18         address_space_rw(cpu->cpu_ases[asidx].as, phys_addr,

```

这两段代码整体是 QEMU 在 **调试模式下完成 guest 虚拟地址访问的处理流程**。

riscv_cpu_get_phys_page_debug 用于将 guest 的虚拟地址翻译为物理地址；调用 **get_physical_address** 获得 **phys_addr** 和访问权限 **prot**，如果翻译成功就返回物理地址，用于后续访问。

cpu_memory_rw_debug 在调试模式下执行实际的内存读写操作。首先根据 **attrs** 获取对应的 address space 索引，然后计算当前页剩余可访问长度 **l**，处理跨页情况；根据是否为写操作调用 **address_space_rw** 读写宿主内存，从而完成虚拟地址到物理地址的安全访问模拟。

我们继续运行，找到下一个 **get_physical_address** 断点并单步调试查看执行逻辑，如下所示

```
Thread 1 "qemu-system-ris" hit Breakpoint 3, get_physical_address
(env=0x5c5c8a575ffd <trace_qemu_mutex_unlock+41>,
physical=0x7ffe2a9d8750, prot=0x5c5cc7756a80,
addr=101552529024910, access_type=23644, mmu_idx=-1972689064)
at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:158
158     {
(gdb) n
163         int mode = mmu_idx;
(gdb) n
165         if (mode == PRV_M && access_type != MMU_INST_FETCH) {
(gdb) n
171         if (mode == PRV_M || !riscv_feature(env, RISCV_FEATURE_
MMU)) {
(gdb) n
177         *prot = 0;
(gdb) n
181         int mxr = get_field(env->mstatus, MSTATUS_MXR);
(gdb) n
183         if (env->priv_ver >= PRIV_VERSION_1_10_0) {
(gdb) n
184             base = get_field(env->satp, SATP_PPN) << PGSHIFT;
(gdb) n
185             sum = get_field(env->mstatus, MSTATUS_SUM);
(gdb) n
186             vm = get_field(env->satp, SATP_MODE);
```



```

(gdb) n
191         levels = 3; ptidxbits = 9; ptesize = 8; break;
(gdb) n
223         CPUState *cs = env_cpu(env);
(gdb) n
224         int va_bits = PGSHIFT + levels * ptidxbits;
(gdb) n
225         target_ulong mask = (1L << (TARGET_LONG_BITS - (va_bits
- 1))) - 1;
(gdb) n
226         target_ulong masked_msbs = (addr >> (va_bits - 1)) & ma
sk;
(gdb) n
227         if (masked_msbs != 0 && masked_msbs != mask) {
(gdb) n
231         int ptshift = (levels - 1) * ptidxbits;
(gdb) n
237         for (i = 0; i < levels; i++, ptshift -= ptidxbits) {
(gdb) n
238             target_ulong idx = (addr >> (PGSHIFT + ptshift)) &
(gdb) n
239                 ((1 << ptidxbits) - 1);
(gdb) n
238             target_ulong idx = (addr >> (PGSHIFT + ptshift)) &
(gdb) n
242             target_ulong pte_addr = base + idx * ptesize;
(gdb) n
244             if (riscv_feature(env, RISCV_FEATURE_PMP) &&
(gdb) n
245                 !pmp_hart_has_privs(env, pte_addr, sizeof(targe
t_ulong),
(gdb) n
244             if (riscv_feature(env, RISCV_FEATURE_PMP) &&
(gdb) n
252             target_ulong pte = ldq_phys(cs->as, pte_addr);
(gdb) n
254             target_ulong ppn = pte >> PTE_PPN_SHIFT;

```

```

(gdb) n
256         if (!(pte & PTE_V)) {
(gdb) n
259         } else if (!(pte & (PTE_R | PTE_W | PTE_X))) {
(gdb) n
262         } else if ((pte & (PTE_R | PTE_W | PTE_X)) == PTE_W
) {
(gdb) n
265         } else if ((pte & (PTE_R | PTE_W | PTE_X)) == (PTE_
W | PTE_X)) {
(gdb) n
268         } else if ((pte & PTE_U) && (mode != PRV_U) &&
(gdb) n
273         } else if (!(pte & PTE_U) && (mode != PRV_S)) {
(gdb) n
276         } else if (ppn & ((1ULL << ptshift) - 1)) {
(gdb) n
279         } else if (access_type == MMU_DATA_LOAD && !(pte &
PTE_R) ||
(gdb) n
283         } else if (access_type == MMU_DATA_STORE && !(pte &
PTE_W)) {
(gdb) n
286         } else if (access_type == MMU_INST_FETCH && !(pte &
PTE_X)) {
(gdb) n
292             (access_type == MMU_DATA_STORE ? PTE_D : 0)
;
(gdb) n
291         target_ulong updated_pte = pte | PTE_A |
(gdb) n
295         if (updated_pte != pte) {
(gdb) n
333         target_ulong vpn = addr >> PGSHIFT;
(gdb) n
334         *physical = (ppn | (vpn & ((1L << ptshift) - 1)
)) << PGSHIFT;

```

```

(gdb) n
337         if ((pte & PTE_R) || ((pte & PTE_X) && mxr)) {
(gdb) n
338             *prot |= PAGE_READ;
(gdb) n
340             if ((pte & PTE_X)) {
(gdb) n
341                 *prot |= PAGE_EXEC;
(gdb) n
345             if ((pte & PTE_W) &&
(gdb) n
346                 (access_type == MMU_DATA_STORE || (pte
& PTE_D))) {
(gdb) n
347                 *prot |= PAGE_WRITE;
(gdb) n
349             return TRANSLATE_SUCCESS;
(gdb) n
353     }
(gdb) n
riscv_cpu_get_phys_page_debug (cs=0x5c5cc7802590,
addr=18446744072637906944)
at /home/osaq/qemu-4.1.1-debug/target/riscv/cpu_helper.c:389
389     if (get_physical_address(&cpu->env, &phys_addr, &prot,
addr, 0, mmu_idx)) {
(gdb) n
392         return phys_addr;
(gdb) n
393     }
(gdb) n
cpu_get_phys_page_attrs_debug (cpu=0x5c5cc7802590,
addr=18446744072637906944, attrs=0x7ffe2a9d88a8)
at /home/osaq/qemu-4.1.1-debug/include/qom/cpu.h:608
608     }

```

成功啦！这是一次基本完整的页表翻译流程，我们整理上面的函数代码流程如下

get_physical_address

```

1  get_physical_address(env=0x5c5c8a575ffd <trace_qemu_mutex_unlock+41>,
2  physical=0x7ffe2a9d8750, prot=0x5c5cc7756a80, addr=101552529024910,
   access_type=23644, mmu_idx=-1972689064)
3  {
4      int mode = mmu_idx;
5      if (mode == PRV_M && access_type != MMU_INST_FETCH) {
6          if (mode == PRV_M || !riscv_feature(env, RISCV_FEATURE_MMU)) {
7              *prot = 0;
8              int mxr = get_field(env->mstatus, MSTATUS_MXR);
9              if (env->priv_ver >= PRIV_VERSION_1_10_0) {
10                 base = get_field(env->satp, SATP_PPN) << PGSHIFT;
11                 sum = get_field(env->mstatus, MSTATUS_SUM);
12                 vm = get_field(env->satp, SATP_MODE);
13                 levels = 3; ptidxbits = 9; ptesize = 8; break;
14             }
15             CPUState *cs = env_cpu(env);
16             int va_bits = PGSHIFT + levels * ptidxbits;
17             target_ulong mask = (1L << (TARGET_LONG_BITS - (va_bits - 1))) - 1;

```

```

17     target_ulong masked_msbs = (addr >> (va_bits - 1)) & mask;
18     if (masked_msbs != 0 && masked_msbs != mask) {
19         int ptshift = (levels - 1) * ptidxbits;
20         for (i = 0; i < levels; i++, ptshift -= ptidxbits) {
21             target_ulong idx = (addr >> (PGSHIFT + ptshift)) &
22                 ((1 << ptidxbits) - 1);
23             target_ulong idx = (addr >> (PGSHIFT + ptshift)) &
24             target_ulong pte_addr = base + idx * ptesize;
25             if (riscv_feature(env, RISCV_FEATURE_PMP) &&
26                 !pmp_hart_has_privs(env, pte_addr, sizeof(target_ulong),
27                 if (riscv_feature(env, RISCV_FEATURE_PMP) &&
28                 target_ulong pte = ldq_phys(cs->as, pte_addr);
29                 target_ulong ppn = pte >> PTE_PPN_SHIFT;
30             if (!(pte & PTE_V)) {
31             } else if (!(pte & (PTE_R | PTE_W | PTE_X))) {
32             } else if ((pte & (PTE_R | PTE_W | PTE_X)) == PTE_W) {
33             } else if ((pte & (PTE_R | PTE_W | PTE_X)) == (PTE_W | PTE_X)) {
34             } else if ((pte & PTE_U) && ((mode != PRV_U) &&
35             } else if (!(pte & PTE_U) && (mode != PRV_S)) {
36             } else if (ppn & ((1ULL << ptshift) - 1)) {
37             } else if (access_type == MMU_DATA_LOAD && !(pte & PTE_R)) {
38             } else if (access_type == MMU_DATA_STORE && !(pte & PTE_W)) {
39             } else if (access_type == MMU_INST_FETCH && !(pte & PTE_X)) {
40                 (access_type == MMU_DATA_STORE ? PTE_D : 0);
41                 target_ulong updated_pte = pte | PTE_A |
42             if (updated_pte != pte) {
43                 target_ulong vpn = addr >> PGSHIFT;
44                 *physical = (ppn | (vpn & ((1L << ptshift) - 1))) << PGSHIFT;
45             if ((pte & PTE_R) || ((pte & PTE_X) && mxr)) {
46                 *prot |= PAGE_READ;
47                 if ((pte & PTE_X)) {
48                     *prot |= PAGE_EXEC;
49                 if ((pte & PTE_W) &&
50                     (access_type == MMU_DATA_STORE || (pte & PTE_D))) {
51                     *prot |= PAGE_WRITE;
52                 return TRANSLATE_SUCCESS;
53             }
54 riscv_cpu_get_phys_page_debug (cs=0x5c5cc7802590,
55 addr=18446744072637906944)
56     if (get_physical_address(&cpu->env, &phys_addr, &prot,
57     return phys_addr;
58 }
59 cpu_get_phys_page_attrs_debug (cpu=0x5c5cc7802590,
60 addr=18446744072637906944, attrs=0x7ffe2a9d8a88)
61 {

```

这里我们得到的代码正是一个完整的软件页表遍历（page table walk）流程，也就是 QEMU 的慢路径（slow path），只在 TLB 未命中 / 调试 / 缺页处理前 才会执行。

具体执行逻辑如下

1、判定当前翻译所用的特权级

代码块

```
1  int mode = mmu_idx;
```

`mmu_idx` 即为当前访问使用的 翻译特权级（PRV_U / PRV_S / PRV_M）

2、判断是否需要 MMU（是否直接 VA=PA）

代码块

```
1  if (mode == PRV_M || !riscv_feature(env, RISCV_FEATURE_MMU)) {
2      *physical = addr;
3      *prot = PAGE_READ | PAGE_WRITE | PAGE_EXEC;
4      return TRANSLATE_SUCCESS;
5  }
```

这一步判断是否要走页表

- M-mode 且非取指 → 不走页表
- 没开启 MMU → 不走页表

3、从 satp 中解析页表结构（Sv39 / Sv48）

代码块

```
1  base = get_field(env->satp, SATP_PPN) << PGSHIFT;
2  vm    = get_field(env->satp, SATP_MODE);
3  levels = 3; ptidxbits = 9; ptesize = 8;
```

通过 `SATP_PPN` 获得根页表的物理地址，读取 `SATP_MODE` 确定虚拟地址模式（如 Sv39），根据规范设置页表层数（3 层）、页表索引位数（9 位）和 PTE 大小（8 字节），初始化页表遍历的基本参数。

4、虚拟地址合法性检查（canonical VA）

代码块

```
1  masked_msbs = (addr >> (va_bits - 1)) & mask;
2  if (masked_msbs != 0 && masked_msbs != mask) {
3      // fault
```

```
4 }
```

检查高位符号扩展**是否符合 Sv39 要求**，即 VA 的高位必须是符号扩展的一致值，如果不合法就触发访问异常，这里未触发说明 VA 合法。

5、核心：逐级页表遍历（page table walk）

代码块

```
1  for (i = 0; i < levels; i++) {
2      idx = (addr >> (PGSHIFT + ptshift)) & ((1 << ptidxbits) - 1);
3      pte_addr = base + idx * ptesize;
4      pte = ldq_phys(cs->as, pte_addr);
```

对每一层页表，根据 VPN[i] 计算页表索引，从物理内存加载对应的 PTE，这是 QEMU 中软件模拟的硬件页表遍历过程，实现虚拟地址到物理地址的转换。

6、所有 PTE 合法性检查

代码块

```
1  if (!(pte & PTE_V)) ...
2  else if (!(pte & (PTE_R | PTE_W | PTE_X))) ...
3  else if ((pte & PTE_U) && (mode != PRV_U)) ...
4  else if (!(pte & PTE_U) && (mode != PRV_S)) ...
5  ...
```

对加载的 PTE 进行有效位（V）、读/写/执行权限（R/W/X）、用户/特权权限（U/S）、页对齐等检查，确保访问合法，不符规则则触发异常，对应 RISC-V Privileged Spec 的规范。

7、A/D 位更新（访问位 / 脏位）

代码块

```
1  updated_pte = pte | PTE_A | (store ? PTE_D : 0);
```

模拟硬件自动设置 PTE 的访问位（A）和写入脏位（D），必要时写回页表，使得 OS 可以通过位更新追踪访问和写操作，这是完整页表访问权限管理的一部分。

8、计算最终物理地址（PA）

代码块

```
1  vpn = addr >> PGSHIFT;
2  *physical = (ppn | (vpn & ((1L << ptshift) - 1))) << PGSHIFT;
```

根据 PTE 的物理页号（PPN）和页内偏移计算最终物理地址，将虚拟页映射到物理内存，完成 **VA → PA 转换**。

9、设置访问权限并返回成功

代码块

```
1  *prot |= PAGE_READ / WRITE / EXEC;  
2  return TRANSLATE_SUCCESS;
```

根据页表权限设置返回的访问权限标志（读/写/执行），并返回 `TRANSLATE_SUCCESS`，表示虚拟地址访问合法，**整个地址转换流程成功完成**。

翻译完成后，最后还出现的这些函数 `riscv_cpu_get_phys_page_debug` 和 `cpu_get_phys_page_attrs_debug`

主要是用于只是封装/返回结果

四、调试模拟cpu查找tlb的C代码

要求：是否能够在qemu-4.1.1的源码中找到模拟cpu查找tlb的C代码，通过调试说明其中的细节。

询问大模型，我们知道了模拟cpu查找tlb的C代码主要是通过 `tlb_entry()` 函数实现的，我们重复上面相似的操作步骤，对该函数打断点，并单步执行查看内部逻辑，最终得到的结果如下


```

Thread 3 "qemu-system-ris" hit Breakpoint 1, tlb_entry (
    env=0x5f44f258bfa0, mmu_idx=1, addr=18446744072637907160)
    at /home/osaq/qemu-4.1.1-debug/include/exec/cpu_ldst.h:180
180      return &env_tlb(env)->f[mmu_idx].table[tlb_index(env, m
mu_idx, addr)];
(gdb) n
181    }
(gdb) n
get_page_addr_code (env=0x5f44f258bfa0, addr=18446744072637907160)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/cputlb.c:1030
1030      if (unlikely(!tlb_hit(entry->addr_code, addr))) {
(gdb) n
1039      if (unlikely(entry->addr_code & (TLB_RECHECK | TLB_MMIO
))) {
(gdb) n
1051      p = (void *)((uintptr_t)addr + entry->addend);
(gdb) n
1052      return qemu_ram_addr_from_host_nofail(p);
(gdb) n
1053    }
(gdb) n
tb_htable_lookup (cpu=0x5f44f2583590, pc=18446744072637907160,
    cs_base=0, flags=24577, cf_mask=4278714368)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/cpu-exec.c:340
340      if (phys_pc == -1) {
(gdb) n
343      desc.phys_page1 = phys_pc & TARGET_PAGE_MASK;
(gdb) n
344      h = tb_hash_func(phys_pc, pc, flags, cf_mask, *cpu->tra
ce_dstate);
(gdb) n
345      return qht_lookup_custom(&tb_ctx.htable, &desc, h, tb_l
ookup_cmp);
(gdb) n
346    }
(gdb) n

```

可以看到主要涉及**tlb_entry()**和**get_page_addr_code()**这两个函数

tlb_entry

```

1  tlb_entry (
2  env=0x5f44f258bfa0, mmu_idx=1, addr=18446744072637907160)
3      return &env_tlb(env)->f[mmu_idx].table[tlb_index(env, mmu_idx, addr)];
4  }
5  get_page_addr_code (env=0x5f44f258bfa0, addr=18446744072637907160)
6      if (unlikely(!tlb_hit(entry->addr_code, addr))) {
7      if (unlikely(entry->addr_code & (TLB_RECHECK | TLB_MMIO))) {
8          p = (void *)((uintptr_t)addr + entry->addend);
9          return qemu_ram_addr_from_host_nofail(p);
10     }

```

`tlb_entry`函数主要用于在 **QEMU 的软件 TLB 表** 中查找指定虚拟地址对应的 TLB 条目。执行逻辑如下：

- 1、`env_tlb(env)` 获取 CPU 所持有的 TLB 结构。
- 2、`f[mmu_idx]`：选择对应的特权级或访问类型的 TLB 表。
- 3、`tlb_index(env, mmu_idx, addr)`：计算虚拟地址在 TLB 表中的索引。
- 4、返回 **TLB 条目指针** (`TlbEntry*`)，包含 guest VA → PA 映射信息、权限、附加偏移等。

`get_page_addr_code()`主要用于将 guest 虚拟地址映射到 **host 内存地址并且**根据 TLB 命中状态决定走 **快路径** 或 **慢路径**。执行逻辑如下：

- 1、`tlb_hit(entry->addr_code, addr)`：检查 TLB 条目是否命中当前访问地址，如果命中，继续快路径返回 host 内存地址。`unlikely()` 是宏，告诉编译器这个分支不常走，优化预测分支。
- 2、如果 TLB 条目标记了 `TLB_RECHECK` 或 `TLB_MMIO`，说明需要重新检查该条目（如跨页、权限变化）或者访问的是内存映射 I/O，此时会将 guest 虚拟地址加上 TLB 条目中记录的 `addend` 偏移量得到对应的 host 内存地址，再通过 `qemu_ram_addr_from_host_nofail()` 转换为 QEMU RAM 内部的 host 指针，从而完成对访存地址的正确映射和访问。

五、qemu中模拟出来的tlb和我们真实cpu中的tlb逻辑上的区别

要求：仍然是tlb，qemu中模拟出来的tlb和我们真实cpu中的tlb有什么**逻辑上的区别**（提示：可以尝试找一条未开启虚拟地址空间的访存语句进行调试，看看调用路径，和开启虚拟地址空间之后的访存语句对比）

相关调试过程我们在前面部分已经涉及了，这里我们就直接整合前面的分析回答结果了，我们主要从存在形式、触发机制、权限检查、慢路径处理四个方面分析。

1. 存在形式

- **真实 CPU TLB：**硬件实现的高速缓存，通常与 MMU 紧密结合，直接缓存虚拟地址到物理地址的映射，并且支持按页大小、多级页表映射等，访问非常快。
- **QEMU TLB：**完全由软件维护的数据结构（数组或哈希表），存储虚拟地址到物理地址的映射。它模拟硬件 TLB 的行为，但本质上只是软件缓存，并不是真正的寄存器或硬件缓存。

2. 触发机制

- **真实 CPU：**当 CPU 执行访存指令时，硬件 MMU 自动查询 TLB，如果 TLB miss，硬件会触发页表遍历或异常处理（如页错误）。
- **QEMU：**每次访存都先通过 `tlb_entry(env, mmu_idx, addr)` 查找软件 TLB，如果未命中就调用 `tlb_fill()`，由它执行慢路径的页表遍历（`get_physical_address()`）并回填 TLB。

也就是说，TLB miss 的处理在 QEMU 是**显式的软件函数调用**，而在硬件中是 CPU 内部自动完成。

3. 权限检查

- **真实 CPU**: TLB 中通常存储有效位和访问权限位 (R/W/X/U/S)，访问时硬件直接检查，违规会触发异常。
- **QEMU**: 软件 TLB 条目也保存权限信息 (如 `prot = PAGE_READ | PAGE_WRITE | PAGE_EXEC`)，但是权限检查是由软件逻辑 (如 `store_helper`、`get_physical_address`) 显式执行的，而非硬件自动完成。

4. 慢路径处理

- **真实 CPU**: TLB miss 后由硬件页表遍历完成，可能触发异常，CPU 内部完成，不暴露给程序。
- **QEMU**: TLB miss 后调用 `tlb_fill` → `riscv_cpu_tlb_fill` → `get_physical_address()`，软件模拟页表遍历和权限检查，并可能触发异常 (通过 `riscv_raise_exception` 和 `riscv_cpu_do_interrupt` 调用链)，再回填 TLB。

注: 下面这两个要求关键部分一部分都结合在上面对调试过程中了，这里再额外列出几个上面没有涉及到的部分。

记录下你调试过程中比较抓马有趣的细节，以及在观察模拟器通过软件模拟硬件执行的时候了解到的知识。

记录实验过程中，有哪些通过大模型解决的问题，记录下当时的情景，你的思路，以及你和大模型交互的过程。

1、Ignoring packet error

```
(gdb) add-symbol-file obj/__user_testbss.out 0x800020
add symbol table from file "obj/__user_testbss.out" at
      .text_addr = 0x800020
(y or n) y
Reading symbols from obj/__user_testbss.out...
(gdb) b load_icode
Breakpoint 1 at 0xfffffffffc02048e2: file kern/process/proc.
c, line 608.
(gdb) c
Continuing.
Ignoring packet error, continuing...
□
```

问题解决: 使用命令 `set remotetimeout unlimited`

2、下断点表示没有找到对应的函数定义

```

(gdb) b cpu_memory_rw
Function "cpu_memory_rw" not defined.
Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 1 (cpu_memory_rw) pending.
(gdb) b cpu_translate
Function "cpu_translate" not defined.
Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 2 (cpu_translate) pending.
(gdb) b tlb_fill
Breakpoint 3 at 0x5e40e7cee195: file /home/osaq/qemu-4.1.1-debug/accel/tcg/cputlb.c, line 870.
(gdb) b riscv_translate
Function "riscv_translate" not defined.
Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 4 (riscv_translate) pending.
(gdb) b riscv_memory_rw
Function "riscv_memory_rw" not defined.
Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 5 (riscv_memory_rw) pending.
(gdb) c

```

我们尝试使用pending挂起断点，但些断点也从来没有被触发，我们删除tlb_fill断点看其他断点是否会被触发后，程序直接运行结束了，没有出现中断

```

Num      Type      Disp Enb Address                               What
1        breakpoint keep y    <PENDING>                          cpu_memory_rw
2        breakpoint keep y    <PENDING>                          cpu_translate
3        breakpoint keep y    0x000063eafd004195 in tlb_fill
                                     at /home/osaq/qe
mu-4.1.1-debug/accel/tcg/cputlb.c:870
      breakpoint already hit 176 times
4        breakpoint keep y    <PENDING>                          riscv_translate
5        breakpoint keep y    <PENDING>                          riscv_memory_rw
6        breakpoint keep y    0x000063eafd004195 in tlb_fill
                                     at /home/osaq/qe
mu-4.1.1-debug/accel/tcg/cputlb.c:870
      stop only if addr < 0x100000000000ULL && mmu_idx < 4
(gdb) delete3
Undefined command: "delete3". Try "help".
(gdb) delete 3
(gdb) c
Continuing.

```

我们询问大模型，它让我们运行命令如 `info functions cpu_memory_rw` 检查函数是否存在，结果如下

```
(gdb) info functions cpu_memory_rw
All functions matching regular expression "cpu_memory_rw":

File /home/osaq/qemu-4.1.1-debug/exec.c:
3964:
    int cpu_memory_rw_debug(CPUState *, target_ulong, uint8_t *, ta
rget_ulong, int);
(gdb)
```

所以断点打不上是因为前面大模型给我们的函数名是错误的，比如cpu_memory_rw在我们的qemu源码中应该是cpu_memory_rw_debug，所以大模型给的回答并不一定都是正确的，我们使用前最好进行进一步的验证。

我们可以通过以下命令查找查找一些相关的函数用于打断点，并将结果给大模型分析筛选，防止大模型给了我们不正确的函数名

```
osaq@LAPTOP-QS28APKV:~$ cd /home/osaq/qemu-4.1.1-debug
osaq@LAPTOP-QS28APKV:~/qemu-4.1.1-debug$ nm -C /home/osaq/qemu-4.1.1-debug/riscv64-softmmu/qemu-system-riscv64 | grep -i tlb
0000000000d0b3a0 D A64TLBI_TLBIMapper
0000000000d0b1a0 d TLBIPairs
0000000000f066c4 B _TRACE_VHOST_IOTLB_MISS_DSTATE
0000000000ec2940 D _TRACE_VHOST_IOTLB_MISS_EVENT
0000000000391f11 t _nocheck_trace_vhost_iotlb_miss
0000000000292e8d t address_space_get_iotlb_entry
000000000029336e T address_space_translate_for_iotlb
0000000000310819 t copy_tlb_helper_locked
000000000030ed38 t env_tlb
00000000003b8148 t env_tlb
00000000003ba3cb t gen_helper_tlb_flush
00000000003c7fb2 T helper_tlb_flush
0000000000298913 T iotlb_to_section
00000000003ff897 t memfd_backend_get_hugetlb
00000000003ffa9f t memfd_backend_get_hugetlbsize
00000000003ff8da t memfd_backend_set_hugetlb
00000000003ff926 t memfd_backend_set_hugetlbsize
0000000000294d49 T memory_region_section_get_iotlb
0000000000c9303 T riscv_cpu_tlb_fill
000000000030f4ac t sizeof_tlb
000000000033ae35 T tcg_flush_softmmu_tlb
00000000002a0cb1 t tcg_out_tlb_load
0000000000310aa5 t tlb_add_large_page
000000000030edfd t tlb_addr_write
```

```
osaq@LAPTOP-QS28APKV:~/qemu-4.1.1-debug$ nm -C /home/osaq/qemu-4.1.1-debug/riscv64-softmmu/qemu-system-riscv64 2>/dev/null | egrep -i 't'
ranslate|tlb|mmu|memory|riscv_translate|riscv_cpu'
0000000000d0b3a0 D A64TLBI_TLBIMapper
00000000007dbf68 t DecodeMemMultipleWritebackInstruction
0000000000cce470 D DumpGuestMemoryFormat_lookup
0000000000cceaf0 D MemoryDeviceInfoKind_lookup
00000000003917b0 t MemoryRegionSection_eq
0000000000d0b1a0 d TLBIPairs
0000000000f05cc2 B _TRACE_LM32_UART_MEMORY_READ_DSTATE
0000000000eb5fc0 D _TRACE_LM32_UART_MEMORY_READ_EVENT
0000000000f05cc0 B _TRACE_LM32_UART_MEMORY_WRITE_DSTATE
0000000000eb5fa0 D _TRACE_LM32_UART_MEMORY_WRITE_EVENT
0000000000f0587e B _TRACE_MEMORY_REGION_OPS_READ_DSTATE
0000000000eb0960 D _TRACE_MEMORY_REGION_OPS_READ_EVENT
0000000000f05880 B _TRACE_MEMORY_REGION_OPS_WRITE_DSTATE
0000000000eb0980 D _TRACE_MEMORY_REGION_OPS_WRITE_EVENT
0000000000f0588a B _TRACE_MEMORY_REGION_RAM_DEVICE_READ_DSTATE
0000000000eb0a20 D _TRACE_MEMORY_REGION_RAM_DEVICE_READ_EVENT
0000000000f0588c B _TRACE_MEMORY_REGION_RAM_DEVICE_WRITE_DSTATE
0000000000eb0a40 D _TRACE_MEMORY_REGION_RAM_DEVICE_WRITE_EVENT
0000000000f05882 B _TRACE_MEMORY_REGION_SUBPAGE_READ_DSTATE
```

3、tlb_fill函数参数总是异常

我们设置了tlb_fill函数断点后，函数参数总是异常

```
(gdb) c
Continuing.
[Switching to Thread 0x78ff6d24c700 (LWP 17163)]

Thread 3 "qemu-system-ris" hit Breakpoint 3, tlb_fill (cpu=0x78ff6d24b7d0,
    addr=103633224089504, size=-1,
    access_type=(MMU_DATA_STORE | MMU_INST_FETCH | unknown: 4294967292),
    mmu_idx=24128, retaddr=255)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/cputlb.c:870
870     {
(gdb)
```

但如果我们在某一步使用单步调试找到的tlb_fill函数参数却是正常的，但中断总是不会停在这里

```
(gdb) si
0x000063eafd0041c3 in tlb_fill (cpu=0x63eb10b3f590,
    addr=18446744072638394384, size=8, access_type=MMU_DATA_LOAD,
    mmu_idx=1, retaddr=138833777745025)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/cputlb.c:871
871     CPUClass *cc = CPU_GET_CLASS(cpu);
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 3, tlb_fill (
    cpu=0x7e44c8a67350, addr=109861248728992, size=-1,
    access_type=(unknown: 3223552000), mmu_idx=25579, retaddr=63)
    at /home/osaq/qemu-4.1.1-debug/accel/tcg/cputlb.c:870
870     {
(gdb) x/10i $rip
=> 0x63eafd004195 <tlb_fill>:     endbr64
```

我们使用条件断点b tlb_fill if mmu_idx < 4（正常的mmu_idx一般是小于4的），程序不发生中断直接运行结束了

结合大模型的回答，我们知道了QEMU中 tlb_fill 会被多种路径调用：很多调用是内部伪访存或异常检查（如边界、特权切换等），此时参数看起来非常异常，比如 addr 很大、mmu_idx 非常大或负值；而正常的 guest 内存访问在快路径下通常被 TCG JIT 内联，不会显式调用 C 层的 tlb_fill，所以条件断点（如 mmu_idx < 4）根本不会触发。单步调试时走慢路径（TLB miss 或调试模式），才会进入 C 层函数，参数正常，因此可以观察到正确访存，但平时运行时快路径绕过了函数，所以中断不会停在这里。

所以为了防止出现这种异常，我们最好还是先找到某条特定的一定会被访问的访存指令，再对它的处理进行分析，而不是直接通过打断点盲目寻找。

4、断点不起作用，断点失效，程序跑飞

我们打下如下断点后，程序没有在这里发生中断，直接运行结束了

```
Reading symbols from bin/kernel...
< q to quit, c to continue without paging--c
The target architecture is set to "riscv:rv64".
Remote debugging using localhost:1234
0x0000000000001000 in ?? ()
(gdb) b kern_entry

Breakpoint 1 at 0xffffffffc0200000: file kern/init/entry.S, line
10.
(gdb) c
Continuing.
Remote connection closed
(gdb)
```



```
Re-allocated 64B obj at 0xfffffffffc034b0e8
Re-allocated 128B obj at 0xfffffffffc034cda8
Re-allocated 32B obj at 0xfffffffffc034a048
Re-allocated 64B obj at 0xfffffffffc034b068
Re-allocated 128B obj at 0xfffffffffc034cca8
Step 5 Passed | Score: 5/7
>>> Step 6: Boundary and error handling test

Warning: free ptr not found in any slab!

Step 6 Passed | Score: 6/7
>>> Step 7: Multi-slab allocation test

Step 7 Passed | Score: 7/7

=== SLUB Full Feature Test Completed | Total Score: 7/7 ===
>>> All tests passed successfully! <<<
```

```
For bug reporting instructions, please see:
<https://github.com/sifive/freedom-tools/issues>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
```

```
For help, type "help".
Type "apropos word" to search for commands related to "word".
Reading symbols from bin/kernel...
< q to quit, c to continue without paging--c
The target architecture is set to "riscv:rv64".
Remote debugging using localhost:1234
0x00000000000001000 in ?? ()
(gdb) b* kern_entry

Breakpoint 1 at 0xfffffffffc0200000: file kern/init/entry.S, line
10.
(gdb) c
Continuing.
```

原因主要是由于 GDB 的断点地址 `0xfffffffffc0200000` 是内核编译时的虚拟地址，而 QEMU 实际启动时 CPU 的 PC 是 `0x00000000000001000`，两者地址不一致，所以断点根本没有命中；同时，QEMU 在没有正确等待 GDB 连接或内核初始化异常退出时，会导致远程连接直接关闭，GDB 无法跟上内核加载过程。

后面我们选择不在这个地方打断点，选择在 `kern_init` 或其他地方避免出现这种情况。

5、调试时出现了僵尸进程

调试时尝试查找当前运行的进程并关闭是出现了如下僵尸进程


```
osaq@LAPTOP-QS28APKV:~/oslab/lab2$ ps -ef | grep qemu
osaq      1901      585  0 05:35 pts/6    00:00:00 [qemu-system-ris
] <defunct>
osaq      2974     2372  0 08:41 pts/4    00:00:00 grep --color=aut
o qemu
```

僵尸进程是子进程已经退出，但父进程未调用 `wait()` / `waitpid()` 获取其退出状态导致的。这里的 `qemu-system-riscv` 作为子进程退出后，其父进程（比如启动它的终端 / 脚本）没处理它的退出状态，所以变成了僵尸进程。我们直接关闭它的父进程即可。